

Task 11

Crop Recommendation Classification Pipeline

Name: Samaira Singh

Registration no: 251090051002

Domain: Software and ML

Repository: https://github.com/samaira-s/Synergy_TP

1. Introduction and Problem Statement

This project addresses a multiclass classification problem: recommending the most suitable crop to plant given a set of soil nutrient and climate readings. Given seven numeric input features — Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall — the goal is to predict one of 22 possible crop labels.

This is a practically motivated problem. In an agricultural decision-support context, a model like this could help a farmer or agronomist choose a crop suited to their land's current soil and climate conditions, rather than relying on guesswork or generic regional norms. Because a wrong recommendation carries a real-world cost (a lost growing season, wasted inputs, or reduced yield), the evaluation approach in this report goes beyond plain accuracy to consider what kind of mistakes the model makes and how costly they are.

2. Dataset Description, Target Classes, and Class Distribution

The dataset (`Crop_recommendation.csv`) contains 2,200 rows and 8 columns: seven numeric input features and one categorical target, label. There are 22 distinct crop classes (e.g., rice, maize, chickpea, coffee, jute, lentil, mothbeans, watermelon), each represented by exactly 100 rows — the dataset is perfectly balanced.

No missing values were found in any column, and all feature ranges were consistent with plausible agronomic values (e.g., pH roughly between 3.5 and 10, rainfall from roughly 20 mm to over 300 mm). Because the classes are perfectly balanced, accuracy and macro-averaged metrics are both meaningful here without needing class-weighting or resampling.

3. Data Preparation and Experimental Approach

The dataset was split into training, validation, and test sets using a 70/15/15 ratio via two sequential calls to `train_test_split`, with `stratify=y` applied at each step to preserve the class balance across all three splits. A fixed `random_state=42` was used throughout to keep the workflow reproducible.

- Training set (70%, ~1,540 rows): used to fit all models.
- Validation set (15%, ~330 rows): used to compare candidate models and select the best one.

- Test set (15%, ~330 rows): held out completely until final evaluation, to give an unbiased estimate of real-world performance.

For distance- and gradient-based models (Logistic Regression, KNN), features were standardized using `StandardScaler` inside a Pipeline, fit only on the training data to avoid data leakage from validation/test into the scaling statistics. Tree-based models (Decision Tree, Random Forest) were trained on raw feature values, since they split on thresholds rather than distances and do not require scaling.

4. Classification Models Used

A majority-class baseline (`DummyClassifier`, `strategy='most_frequent'`) was trained first, to establish the minimum performance any real model must clearly exceed to be considered useful. Four real classification models were then trained and compared:

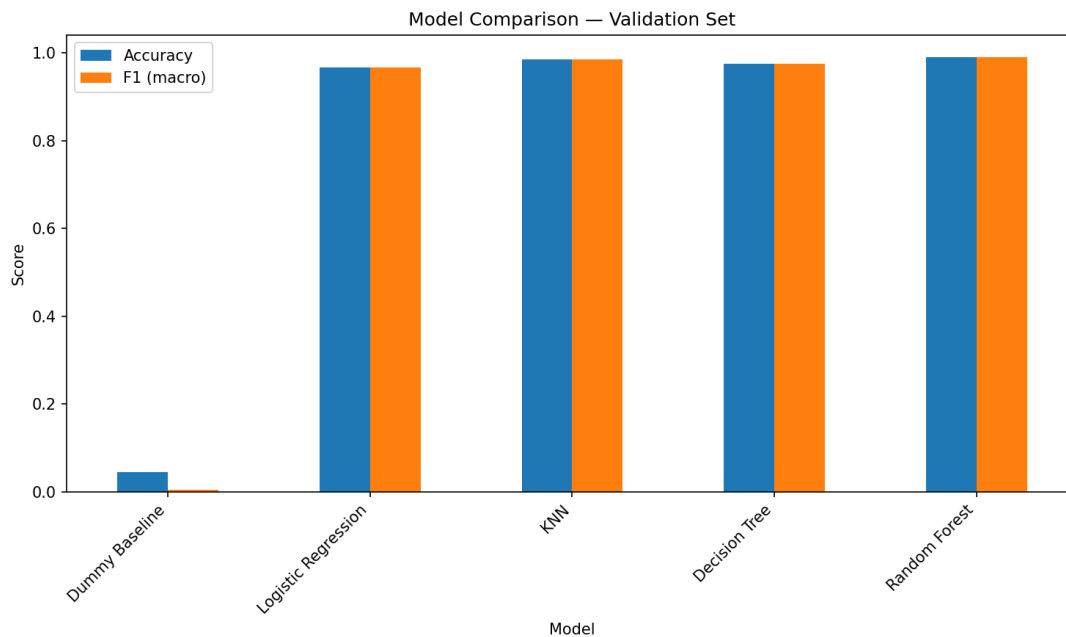
- Logistic Regression — a linear model, fast and interpretable, used as a baseline 'real' model.
- K-Nearest Neighbors (KNN, `k=5`) — a non-linear, instance-based model that classifies a point based on its closest neighbors in feature space.
- Decision Tree (`max_depth=10`) — a single tree, depth-limited to reduce overfitting to the training data.
- Random Forest (200 trees) — an ensemble of decision trees trained on random subsets of data and features, combined by majority vote.

5. Evaluation Metrics and Model-Comparison Table

Two metrics were used to evaluate every model on the validation set: accuracy (overall proportion of correct predictions) and macro-averaged F1 score (the unweighted average of per-class F1 scores across all 22 classes). Macro F1 was chosen alongside accuracy because it treats every class as equally important regardless of how many samples it has — this guards against a model's performance on common classes hiding weak performance on rarer ones. Since this dataset is perfectly balanced, macro and weighted F1 would be very similar in this case, but macro remains the more defensible general default for multiclass problems.

Model	Validation Accuracy	Validation F1 (macro)
Dummy Baseline	4.55%	0.0040
Logistic Regression	96.67%	0.9665
KNN (k=5)	98.48%	0.9848

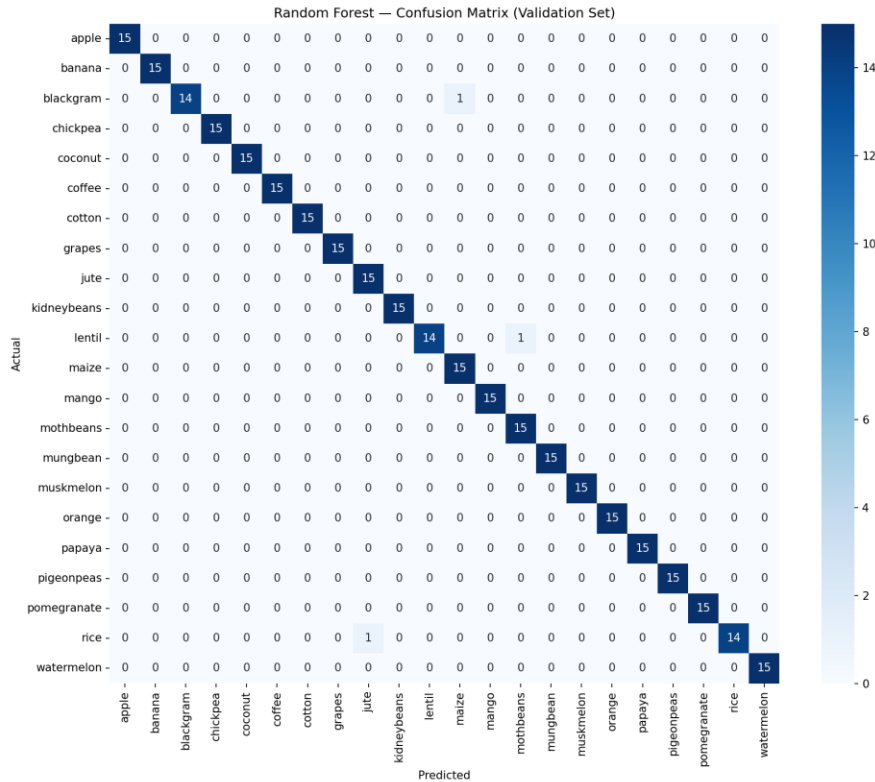
Decision Tree (max_depth=10)	97.58%	0.9754
Random Forest (200 trees)	99.09%	0.9909



All four real models comfortably outperform the dummy baseline (4.55% accuracy), confirming that the seven input features carry strong, learnable signal for distinguishing between crops. Random Forest achieved the highest scores on both metrics.

6. Confusion Matrix and Error Analysis

The confusion matrix below shows Random Forest's predictions against actual labels on the validation set (330 rows, 15 per class).



Out of 330 validation predictions, Random Forest made only 3 errors (99.09% accuracy):

- rice → predicted as jute
- lentil → predicted as mothbeans
- blackgram → predicted as maize

These misclassifications are informative rather than random. Rice and jute are both high-water, high-humidity crops with overlapping climate profiles, and lentil/mothbeans are both pulse (legume) crops with similar nutrient and rainfall requirements — the model's confusion follows genuine agronomic similarity rather than an arbitrary mistake. The blackgram/maize confusion is a smaller outlier but still involves crops that can share moderate nitrogen and rainfall ranges.

This distinction matters for the intended use case: confusing two agronomically similar crops is a low-cost error (the recommended crop may still be a reasonable, if suboptimal, choice for those conditions), whereas confusing two crops with very different growing requirements (e.g., recommending an arid-climate crop for a high-rainfall field) would be a high-cost error with a real risk of crop failure. None of Random Forest's three validation errors fall into this high-cost category.

7. Probability / Threshold Analysis

This is a 22-class multiclass problem rather than a binary one, so a single decision threshold (as used in binary classification to trade off precision and recall) does not directly apply here. Instead, Random Forest's `predict_proba()` output was used to report a confidence score alongside each prediction — the proportion of the ensemble's 200 trees that voted for the predicted class. This is exposed directly in the inference script (`predict.py`), giving a practical substitute for threshold analysis: predictions with low confidence (e.g., below ~70%) could be flagged for manual review in a real deployment, while high-confidence predictions can be trusted more directly.

8. Final Model Selection and Justification

Random Forest was selected as the final model. It achieved the highest validation accuracy (99.09%) and macro F1 (0.9909) among all candidates, and its validation errors were both rare and agronomically low-cost. As an ensemble of 200 decision trees, it is also naturally more robust to overfitting than a single Decision Tree, since individual trees' errors tend to cancel out during majority voting. While KNN performed similarly well (98.48%), Random Forest was preferred for its slightly stronger metrics and because it does not require the input features to be scaled at inference time, simplifying the deployment pipeline.

9. Model Saving and Inference Workflow

After model selection, Random Forest was retrained on the combined training and validation sets (85% of the data) to make use of the additional data before deployment, while the test set remained untouched to preserve it as an unbiased final evaluation set. The retrained pipeline was saved using `joblib` as `crop_recommendation_model.joblib`.

A standalone script, `predict.py`, loads this saved pipeline and accepts seven command-line arguments (N, P, K, temperature, humidity, ph, rainfall). It returns both the predicted crop and a confidence score derived from `predict_proba()`. Example usage:

```
python predict.py 90 42 43 20.8 82.0 6.5 202.9
Output: Recommended crop: rice | Confidence: 97.50%
```

This was verified by re-loading the saved model in a fresh process and confirming it reproduced the same prediction as the in-notebook model on a known sample.

10. Final Test Set Evaluation

As the final and only evaluation on the held-out test set, Random Forest achieved:

Metric	Test Set Result
Accuracy	99.70%
F1 (macro)	0.9970

This is consistent with (and marginally higher than) the validation performance, indicating the model generalizes well and was not overfit to the validation split during model selection.

11. Limitations and Possible Improvements

- Dataset size and diversity: 2,200 rows across 22 classes (100 each) is relatively small per class; real-world soil/climate conditions vary more continuously than this dataset may capture, e.g., regional soil types or seasonal variation not represented here.
- No threshold-based deferral mechanism: while confidence scores are available via `predict_proba()`, the current pipeline does not automatically flag or defer low-confidence predictions for manual review — a useful addition for real deployment.
- Hyperparameters were not extensively tuned: Random Forest and Decision Tree used reasonable but not exhaustively searched settings (e.g., `n_estimators=200`, `max_depth=10`); a grid or randomized search could potentially improve results further, though gains would likely be marginal given the already high performance.
- Feature scope: the model only uses 7 numeric features; real agronomic decisions might also consider soil texture, historical yield data, or market price, which are outside this dataset's scope.

12. Conclusion

This task built a complete multiclass classification pipeline for crop recommendation, from data exploration through model comparison, error analysis, and deployment. Four models were trained and benchmarked against a majority-class baseline, confirming that all four extracted real signal from the input features. Random Forest was selected as the final model based on its superior validation accuracy (99.09%) and macro F1 (0.9909), and it went on to achieve 99.70% accuracy on the completely held-out test set. Beyond raw accuracy, this task illustrated why accuracy alone is not sufficient for model selection: the confusion

matrix and error analysis showed that even the rare errors this model does make tend to occur between agronomically similar crops rather than dissimilar ones, which is an important property for a system intended to support real planting decisions.